Practical No. 11

Aim: To create a Lambda function which will log “An image has been added” once you add an object to a specific bucket in s3.

Objective: To introduce learners to serverless computing concepts and platforms (e.g., AWS Lambda) and the importance of continuous monitoring using tools like Nagios.

Outcome: Build and deploy serverless environment using AWS Lambda and monitor it with Nagios, applying engineering tools to solve complex problems and fostering lifelong learning in evolving cloud technologies

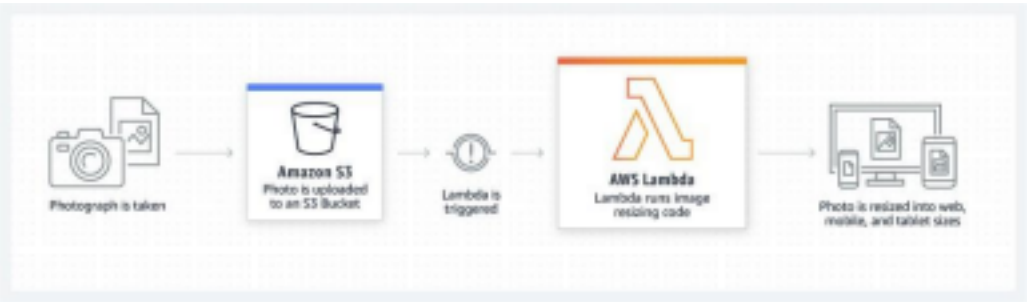
Theory :

AWS Lambda is a serverless compute service that runs your code in response to events and automatically manages the underlying compute resources for you. You can use AWS Lambda to extend other AWS services with custom logic, or create your own back end services that operate at AWS scale, performance, and security. AWS Lambda can automatically run code in response to multiple events, such as HTTP requests via Amazon API Gateway, modifications to objects in Amazon S3 buckets, table updates in Amazon DynamoDB, and state transitions in AWS Step Functions.

What is a Lambda function?

The code you run on AWS Lambda is called a “Lambda function.” After you create your Lambda function, it is always ready to run as soon as it is triggered, similar to a formula in a spreadsheet. Each function includes your code as well as some associated configuration information, including the function name and resource requirements. Lambda functions are “stateless”, with no affinity to the underlying infrastructure, so that Lambda can rapidly launch as many copies of the function as needed to scale to the rate of incoming events.

After you upload your code to AWS Lambda, you can associate your function with specific AWS resources, such as a particular Amazon S3 bucket, Amazon DynamoDB table, Amazon Kinesis stream, or Amazon SNS notification. Then, when the resource changes, Lambda will execute your function and manage the compute resources as needed to keep up with incoming requests.



What is Amazon S3 bucket?

An Amazon S3 bucket is a public cloud storage resource available in Amazon Web Services' (AWS) Simple Storage Service (S3), an object storage offering. Amazon S3 buckets, which are similar to file folders, store objects, which consist of data and its descriptive metadata.

S3 bucket features:

AWS offers several features for Amazon S3 buckets. An IT professional can enable versioning for S3 buckets to preserve every version of an object when an operation is performed on it, such as a copy or delete operation. This helps an IT team prevent accidental deletion of an object. Likewise, upon bucket creation, a user can set up server access logs, object-level API logs, tags and encryption. Also, S3 Transfer Acceleration helps execute fast, secure transfers from a client to an S3 bucket via AWS edge locations.

What are triggers?

A trigger is a Lambda resource or a resource in another service that you configure to invoke your function in response to lifecycle events, external requests, or on a schedule. Your function can have multiple triggers. Each trigger acts as a client invoking your function independently. Each event that Lambda passes to your function only has data from one client or trigger.

What are log files?

A log stream appears when you update your Lambda function, and when additional instances are created to handle multiple concurrent invocations. To find logs for a specific invocation, we recommend instrumenting your function with AWS X-Ray. X-Ray records details about the request and the log stream in the trace.

It keeps track of every event that happens in your application from the minute you start running it to the second you stop it. Any calls you make to third party APIs or any scripts that run in the background will have a record here. This is your source for finding everything that happens behind the scenes of your application.

Why are log files important?

The reason we need logs is because they hold information that can't be found anywhere else. An error will be recorded in the logs so that only someone with access to the server could see those kinds of errors. Most of the time this is where you should look when you can't figure out what's wrong with your code after hours of debugging. The answer might not always be here, but it will give you another place to go check.

Once you start looking in the log files when you have weird errors, it becomes easier to find ways to fix them. At the minimum, you will rule out another place to look. A lot of new developers don't know about logs so it's important that we take the time to teach them so they

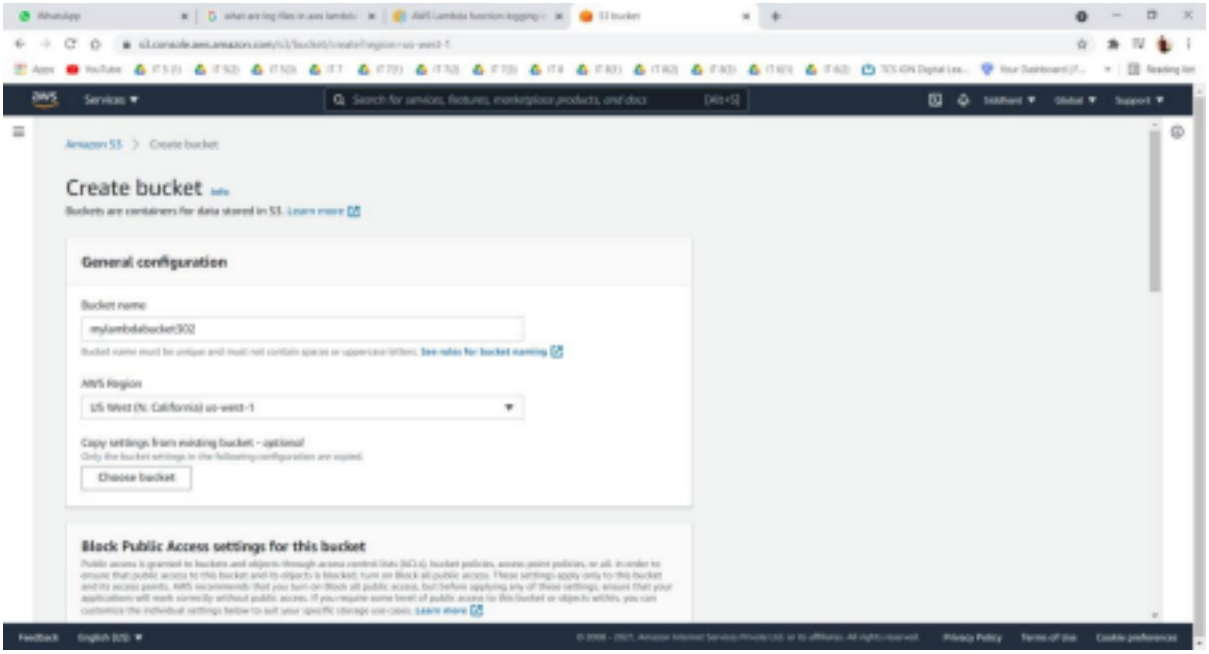
can learn how to better research bugs.

 Create a bucket and upload a sample object:

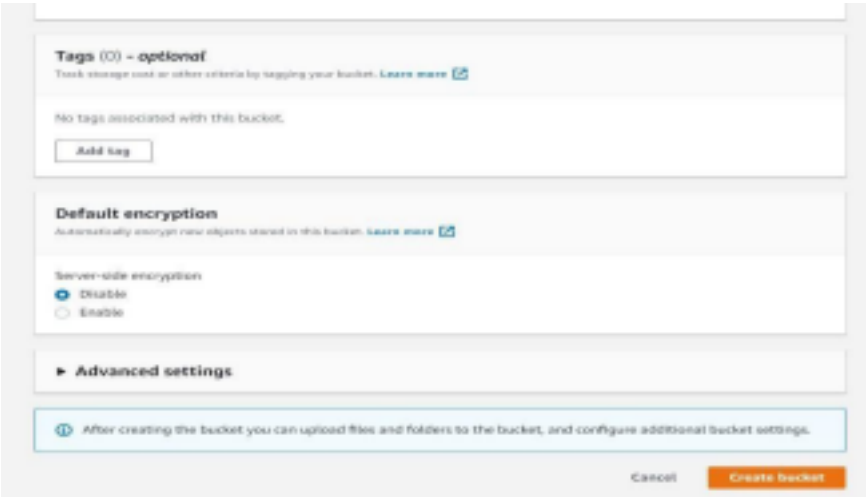
Create an Amazon S3 bucket and upload a test file to your new bucket. Your Lambda function retrieves information about this file when you test the function from the console.

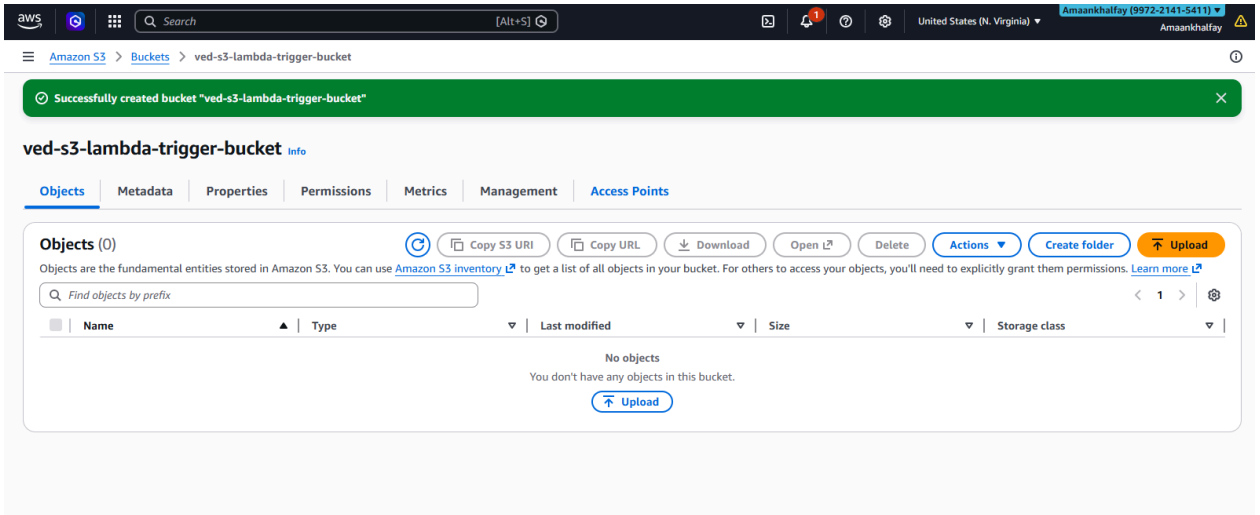
▪ **To create an Amazon S3 bucket using the console**

1. Open the Amazon S3 console.
2. Choose **Create bucket**.
3. Under **General configuration**, do the following:
   - For **Bucket name**, enter a unique name.
   - For **AWS Region**, choose a Region. Note that you must create your Lambda function in the same Region.



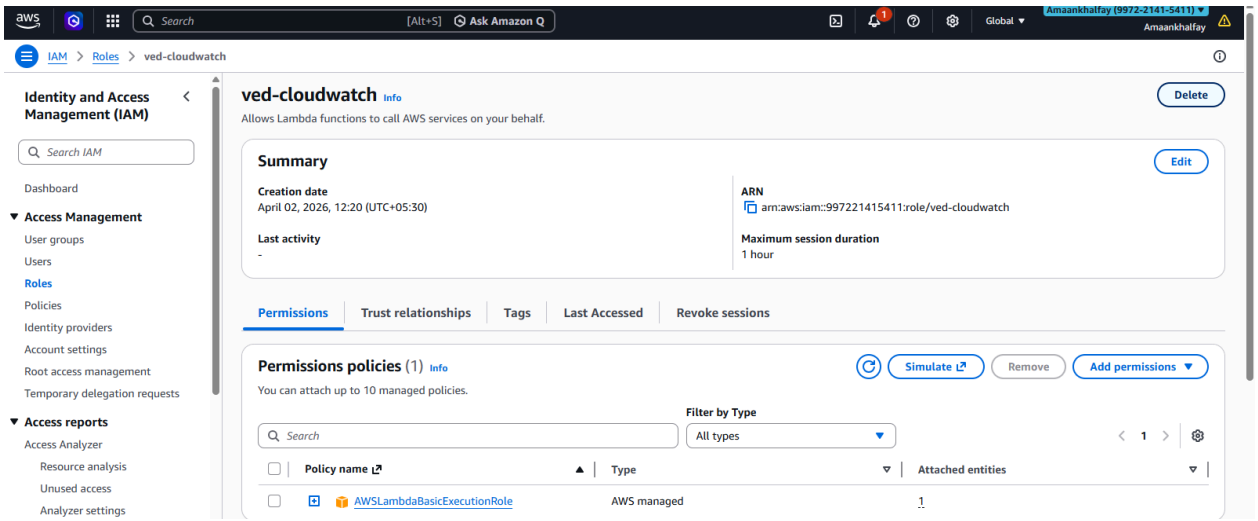
4. Choose **Create bucket**.





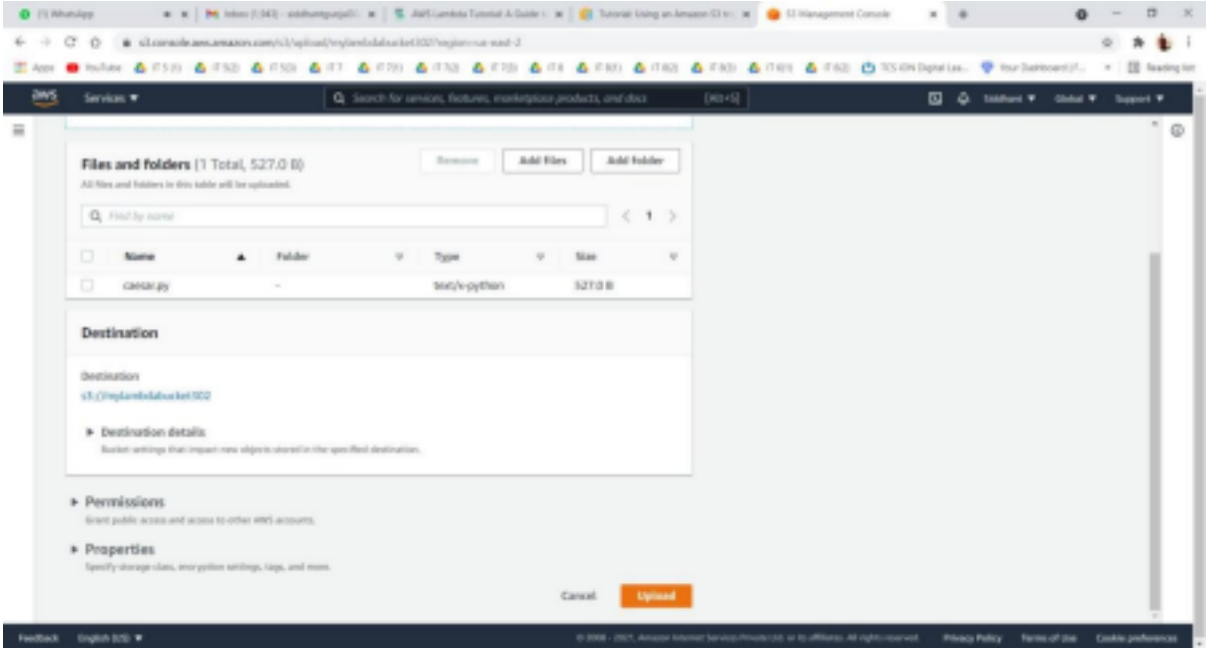
After creating the bucket, Amazon S3 opens the **Buckets** page, which displays a list of all buckets in your account in the current Region.

▪ To create Iam Role and grant permissions in cloudwatch



▪ To upload a test object using the Amazon S3 console

1. On the Buckets page of the Amazon S3 console, choose the name of the bucket that you created.
2. On the **Objects** tab, choose **Upload**.
3. Drag a test file from your local machine to the **Upload** page.



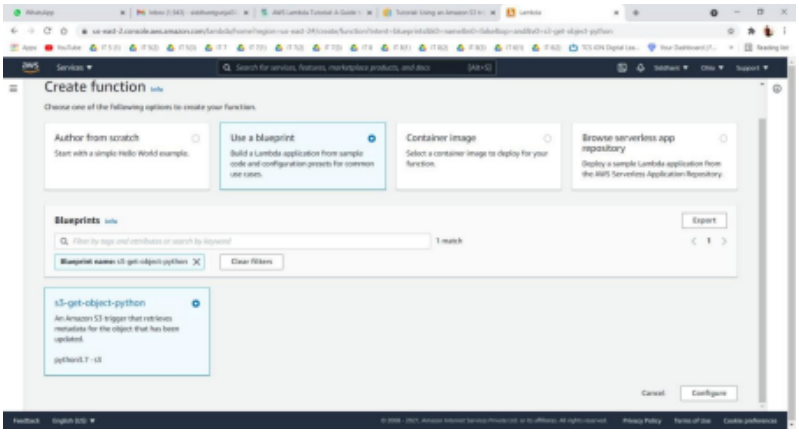
4. Choose **Upload**.

 Create the Lambda function

Use a function blueprint to create the Lambda function. A blueprint provides a sample function that demonstrates how to use Lambda with other AWS services. Also, a blueprint includes sample code and function configuration presets for a certain runtime.

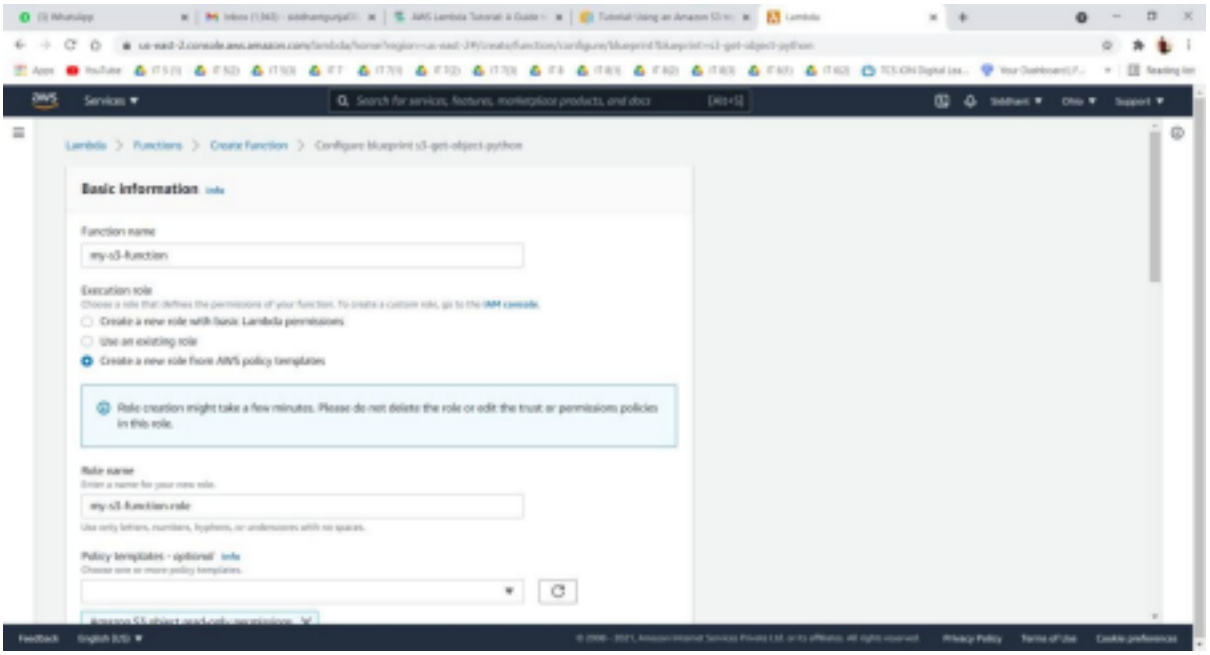
▪ **To create a Lambda function from a blueprint in the console**

1. Open the Functions page on the Lambda console.
2. Choose **Create function**.
3. On the **Create function** page, choose **Use a blueprint**.
4. Under **Blueprints**, enter **s3** in the search box.
5. In the search results, do the following:
   - For a Python function, choose **s3-get-object-python**.
6. Choose **Configure**.

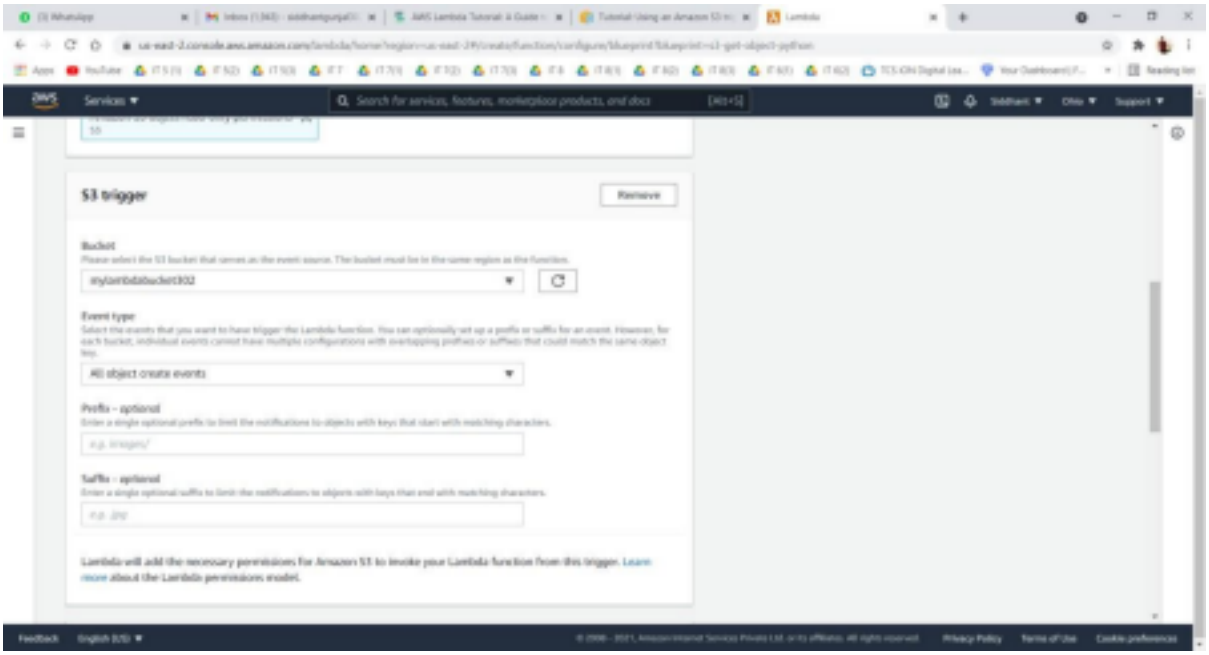


7. Under **Basic information**, do the following:

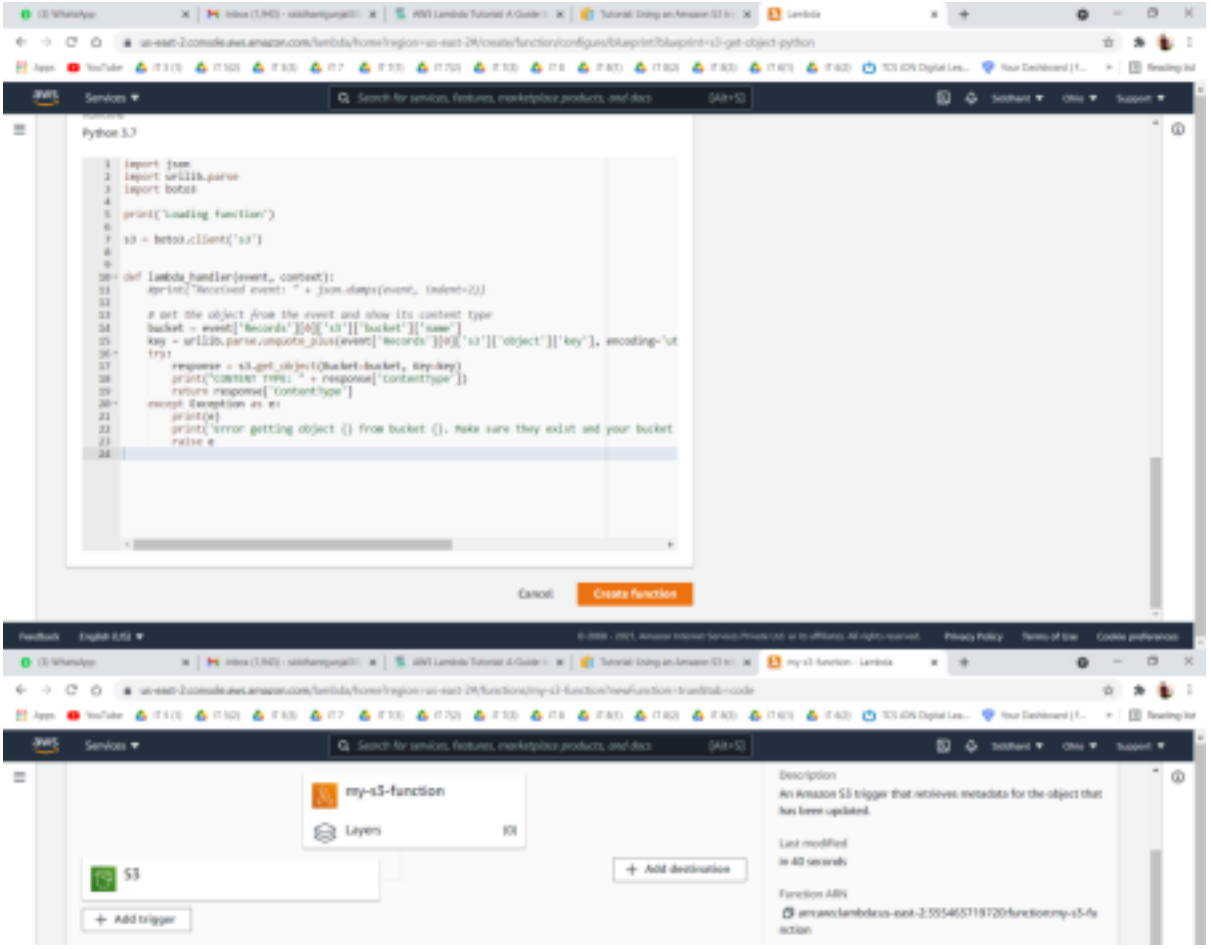
• For **Function name**, enter **my-s3-function**.
• For **Execution role**, choose **Create a new role from AWS policy templates**. •
For **Role name**, enter **my-s3-function-role**.



8. Under **S3 trigger**, choose the S3 bucket that you created previously.



When you configure an S3 trigger using the Lambda console, the console modifies your function's resource-based policy to allow Amazon S3 to invoke the function. 9. Choose **Create function**.

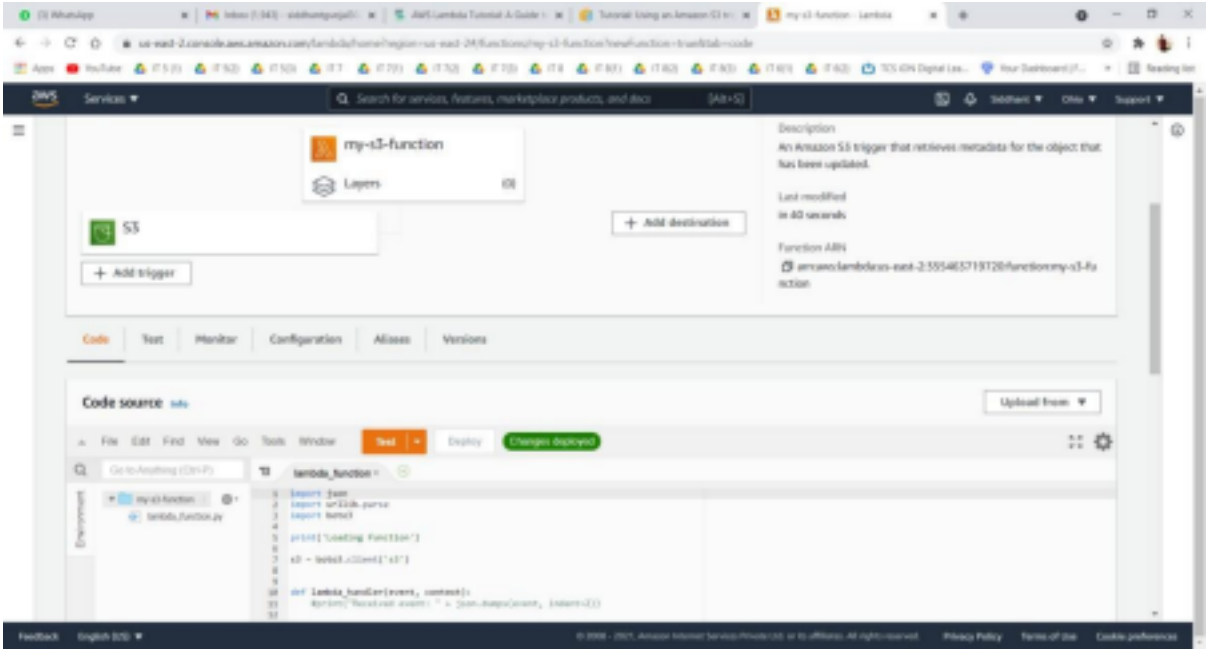


 Test in the console:

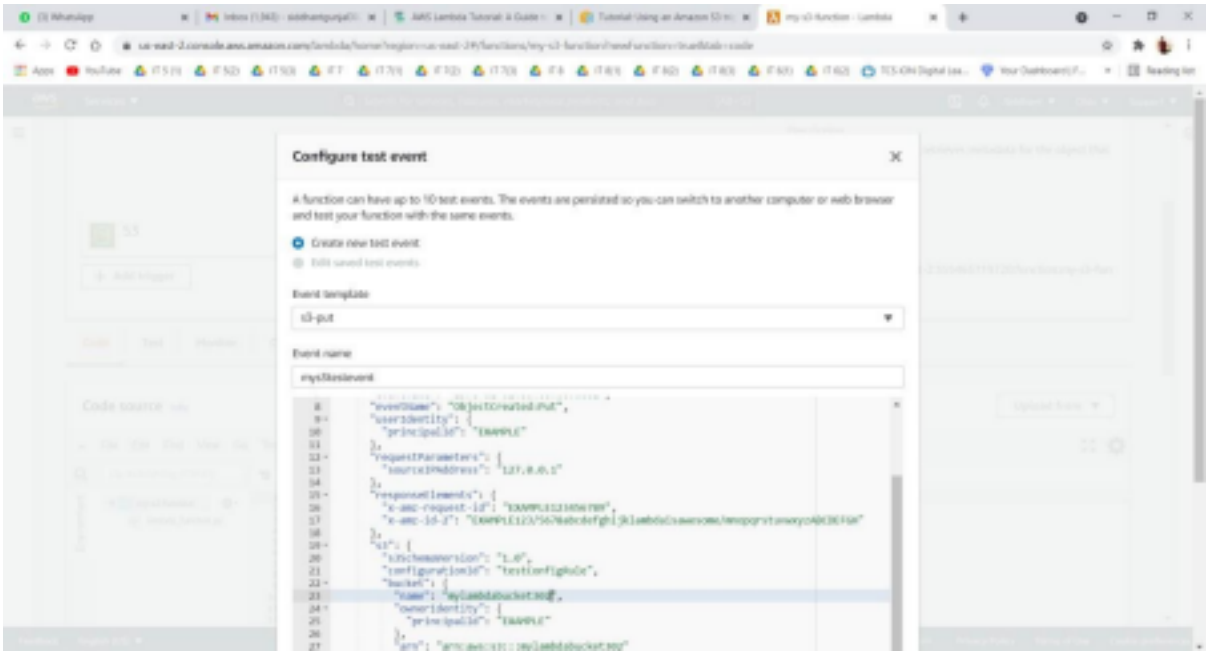
Invoke the Lambda function manually using sample Amazon S3 event

data. ▪ **To test the Lambda function using the console**

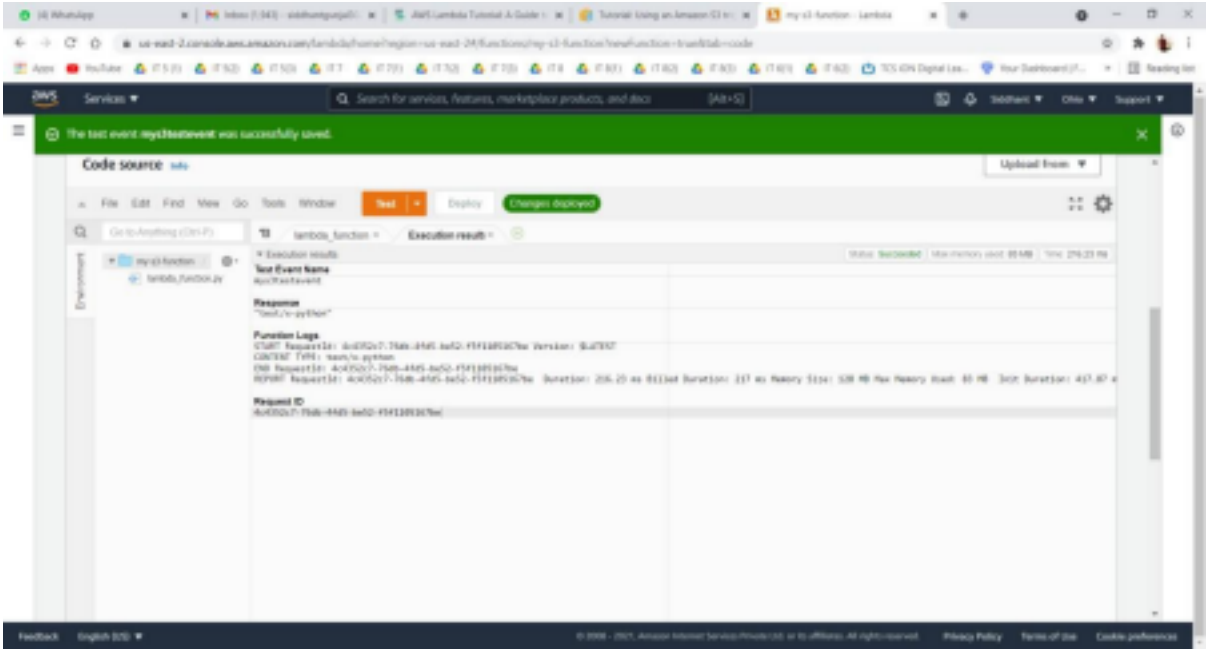
1. On the **Code** tab, under **Code source**, choose the arrow next to **Test**, and then choose **Configure test events** from the dropdown list.



2. In the **Configure test event** window, do the following:
   - Choose **Create new test event**.
   - For **Event template**, choose **Amazon S3 Put** (**s3-put**).
   - For **Event name**, enter a name for the test event. For example, **mys3testevent**.



- Choose **Create**.

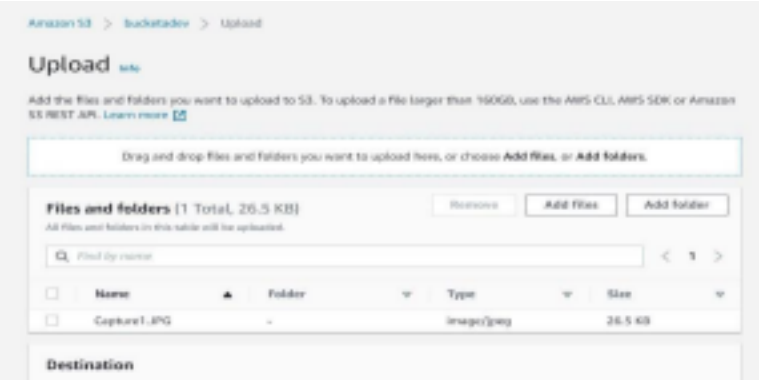


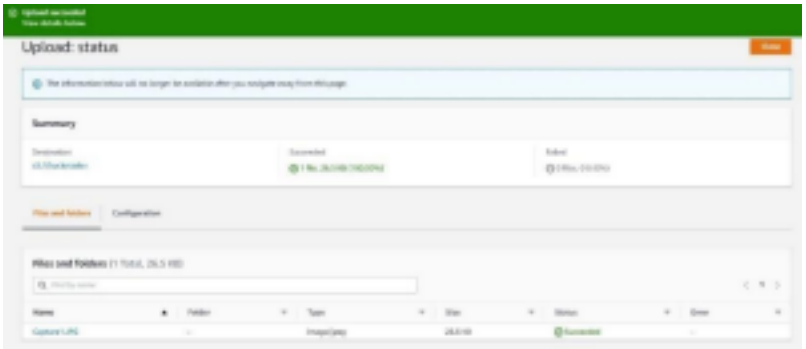
Test with the S3 trigger

Invoke your function when you upload a file to the Amazon S3 source bucket.

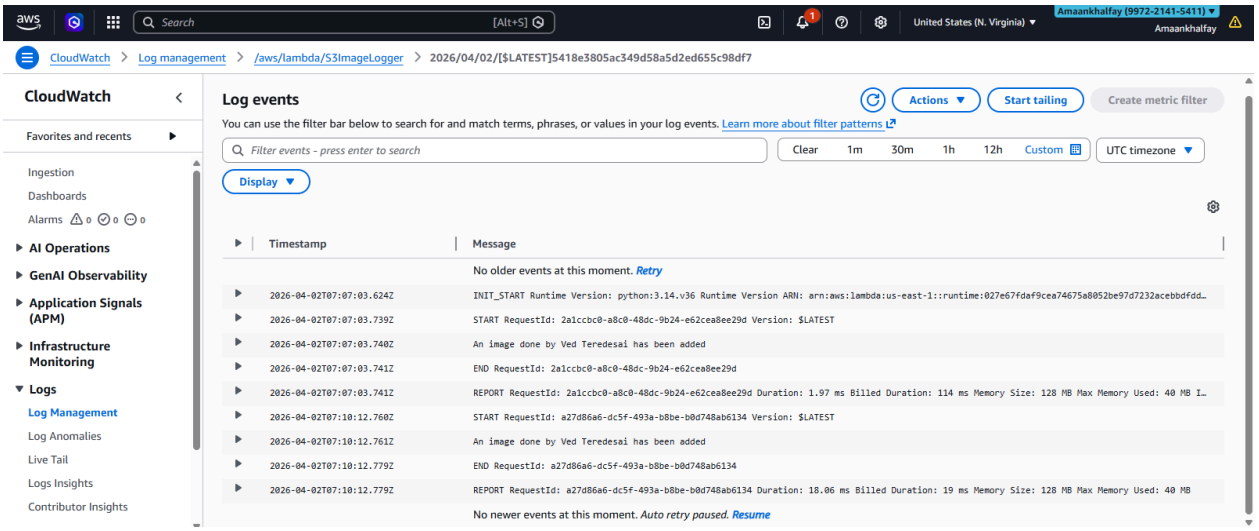
▪ To test the Lambda function using the S3 trigger

1. On the Buckets page of the Amazon S3 console, choose the name of the source bucket that you created earlier.
2. On the **Upload** page, upload a few .jpg or .png image files to the bucket.





3. Open the Functions page on the Lambda console.
4. Choose the name of your function (**my-s3-function**).



Conclusion:

Thus, successfully we created a Lambda function which logged “An image has been added” once we add an object to a specific bucket in s3.